Reviewer Pack — Lyndon Factor Count Separates MOD_3 from Depth-2 ACC^0[2]

Entry 2243c6c24ab3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 10:31:26 UTC

Lyndon Factor Count Separates MOD_3 from Depth-2 ACC^0[2]

Entry ID: `2243c6c24ab3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 10:31:26 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics on words (Chen-Fox-Lyndon factorization, Duval's linear-time algorithm) **Field B** (complexity object): ACC^0[2] circuit complexity, specifically depth-2 XOR-of-AND circuits (the canonical Razborov-Smolensky target where MOD_p vs MOD_q lower bounds are proven)

Statement:

> Let L(T) denote the length of the unique Chen-Fox-Lyndon factorization of a binary string T, computable in linear time via Duval's algorithm. For every depth-2 ACC^0[2] circuit C on n variables of size at most n^2 (top XOR gate over AND gates of width at most ceil(log_2 n)+2), the truth table TT(C) in {0,1}^{2^n} read in lexicographic order satisfies L(TT(C)) <= (2^n)/sqrt(n). In contrast, the explicit family MOD_3_n(x) = [popcount(x) mod 3 == 0] satisfies L(TT(MOD_3_n)) >= (2^n)/8 for all n >= 6, and the gap (2^n)/8 vs (2^n)/sqrt(n) constitutes a Lyndon-witness of MOD_3 not in depth-2-ACC^0[2]_{n^2}.

Rationale (proposer's reasoning):

> Depth-2 XOR-of-AND circuits compute F_2-affine-piecewise functions whose truth tables decompose along F_2-cosets, producing strongly self-similar binary strings that Lyndon factorization compresses into few large runs (Thue-Morse-like behavior dominates). MOD_3 has inherent period-3 ternary structure incompatible with any small F_2-affine cover, so its truth table fragments into many short incomparable Lyndon factors. The Chen-Fox-Lyndon decomposition has not, to our knowledge, been used as a quantitative ACC^0 invariant, even though it is one of the cheapest non-trivial word-combinatorial statistics available.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `61494d3b9bd642da`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For n in {8,10,12} with 30 seeds each (90 random depth-2 ACC^0[2] circuits total), measure L(TT(C)) via Duval. Conjecture supported iff every circuit yields L(TT(C)) <= 2^n/sqrt(n) AND L(TT(MOD_3_n)) >= 2^n/8 for all three n. Any violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Lyndon factorization circuit complexity lower bound truth table` - `Chen-Fox-Lyndon AC0 MOD_p Razborov-Smolensky` - `Duval factorization depth-2 XOR AND circuit MOD 3`

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/1610.03808v2] Lyndon word decompositions and pseudo orbits on q-nary graphs - [http://arxiv.org/abs/1602.01530v8] Randomness Extraction in AC0 and with Small Locality - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/2602.24220v1] Comparing Classical and Quantum Variational Classifiers on the XOR Problem - [http://arxiv.org/abs/1207.1611v1] Nonparametric estimation of a renewal reward process from discrete data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def duval_algorithm(s):
    n = len(s)
    i, j, k = 0, 1, 2
    factors = []

    while i < n:
        if j >= n or s[i] < s[j]:
            factors.append(j - i)
            i = j
            j = k
            k += 1
        elif s[i] > s[j]:
            i = max(i + 1, k)
```

```
            j = k
            k += 1
        else:
            l = i
            while l < j and s[l] == s[j]:
                l += 1
            if l >= j:
                factors.append(j - i)
                i = j
                j = k
                k += 1
            else:
                d = j - l
                for m in range(i, l):
                    s[m], s[m + d] = s[m + d], s[m]
                i += d
                j += d

    factors.append(n - i)
    return factors

def generate_and_circuit(n):
    log_n = math.ceil(math.log2(n))
    and_gates = [random.sample(range(1, n+1), log_n + 2) for _ in range(n**2)]
    circuit = []
    for gate in and_gates:
        circuit.append(gate)
    return circuit

def generate_mod_3_circuit(n):
    def popcount(x):
        count = 0
        while x:
            count += x & 1
            x >>= 1
        return count

    mod_3_circuit = [popcount(i) % 3 == 0 for i in range(2**n)]
    return mod_3_circuit

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12]
    results = []

    for n in n_values:
        circuit = generate_and_circuit(n)
        tt_circuit = [circuit[i] for i in range(2**n)]
        lyndon_factors_circuit = duval_algorithm(''.join(str(bit) for bit in tt_circuit))

        mod_3_circuit = generate_mod_3_circuit(n)
        lyndon_factors_mod_3 = duval_algorithm(''.join(str(bit) for bit in mod_3_circuit))

        results.append({
            "n": n,
```

```
            "lyndon_factors_circuit": len(lyndon_factors_circuit),
            "lyndon_factors_mod_3": len(lyndon_factors_mod_3)
        })

    mean_lyndon_factors_circuit = sum(result["lyndon_factors_circuit"] for result in results) / le
    mean_lyndon_factors_mod_3 = sum(result["lyndon_factors_mod_3"] for result in results) / len(re

    conjecture_holds = all(result["lyndon_factors_circuit"] <= 2**n / math.sqrt(n) and
                           result["lyndon_factors_mod_3"] >= 2**n / 8 for result in results)

    return {
        "metric_name": "Lyndon Factor Count",
        "metric_value": mean_lyndon_factors_circuit,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54f36a74.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54f36a74.py", line 79, in run_trial
    tt_circuit = [circuit[i] for i in range(2**n)]
                  ~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError before producing any data, so neither the support nor the falsification condition was evaluated. Per the rules, a crashed test yields INCONCLUSIVE. | next: Fix the circuit truth-table construction (ensure the circuit object is indexable over the full 2^n domain) and rerun the pre-registered protocol for n in {8,10,12}.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	231667
2	preregistration	claude_max	opus	0	0	5795
3	novelty	claude_max	opus	0	0	2852
4	novelty	claude_max	opus	0	0	8695
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11101
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9322
7	judge	claude_max	opus	0	0	4911

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 274343 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/2243c6c24ab3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2243c6c24ab3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2243c6c24ab3.tar.gz` (if generated)